



TECHNICAL DECISION BASELINE

Architecture Decision Baseline

Final resolved architectural decisions, environment isolation policies, and canonical technology stack baselines.



DOCUMENT ID
ADB-AIWF-2026-001

VERSION
v1.1.0

DATE
October 2026

DOCUMENT CONTROL

Field	Details
Document Title	Architecture Decision Baseline — AI Workforce Platform
Document ID	ADB-AIWF-2026-001
Version	1.0.0
Status	FINAL — Approved for Architecture Phase
Classification	Confidential — Internal Engineering Use
Created Date	2026-10-07
Linked BRD	BRD-AIWF-2026-001 v1.0.0
Linked USC	USC-AIWF-2026-001 v1.0.0
Prepared By	Product Architecture Team
Approved By	Architecture Review Board

Document Purpose

This document resolves all critical open decisions identified in BRD-AIWF-2026-001 Section 31 and supplementary ambiguous requirements. Every decision herein is **final** and forms the authoritative baseline that the following engineering disciplines must follow:

- System Architecture
- Database Design
- API Design
- AI/Agent Architecture
- Security Engineering
- Infrastructure Engineering
- Testing Strategy
- Deployment Strategy

No architectural decision may deviate from this baseline without a formal Architecture Decision Record (ADR) amendment and approval from the Architecture Review Board.

Table of Contents

Decision 1 — AI Model Hosting Strategy (BYOK vs. Managed)

Decision 2 — Multi-Tenant Data Isolation Model

Decision 3 — Vector Database Selection

Decision 4 — MVP AI Evaluation Strategy

Decision 5 — Plan Tier and Usage Limit Strategy

Decision 6 — Phase 2 Connector Strategy

Decision 7 — Approval “Request Changes” Behavior

Decision 8 — Agent Memory Strategy

Decision 9 — Workflow Idempotency Strategy

Decision 10 — Token Cost Calculation Method

Decision 11 — Real-Time Execution Streaming

Decision 12 — Approval Email Action Model

Decision 13 — Data Retention Defaults

Architecture Decision Baseline Table

Canonical System Boundary Definitions

DECISION 1 — AI MODEL HOSTING STRATEGY

Decision Statement

Determine whether the platform manages AI model API credentials on behalf of all organizations (“Managed”) or requires organizations to supply their own API credentials (“BYOK — Bring Your Own Key”), or a hybrid of both.

Recommended Decision: HYBRID MODEL — BYOK Primary, Optional Platform-Managed Keys

FINAL DECISION

The platform adopts a **Hybrid AI Model Strategy** with two modes:

- 1. BYOK (Bring Your Own Key) — Primary Mode:** Organizations configure their own AI model provider API credentials in their organization settings. These credentials are stored in the platform’s encrypted credential vault and used exclusively for that organization’s workloads.
- 2. Platform-Managed Keys — Optional Onboarding Tier:** The platform maintains a small pool of managed API keys exclusively for the **Free Trial / Starter plan tier** to reduce onboarding friction. This is metered: usage is charged back to the organization through the platform’s billing system. Platform-managed key access is removed when an organization upgrades to Professional or Enterprise.

WHY THIS IS THE BEST CHOICE

Factor	Reasoning
Onboarding Friction	New organizations can try the platform without sourcing AI credentials, dramatically reducing time-to-value
Enterprise Readiness	Enterprise customers strongly prefer BYOK for data privacy, compliance, and cost control; forcing managed keys would block enterprise sales
Cost Neutrality	BYOK means AI model costs are borne directly by the organization; the platform does not carry uncapped AI cost liability at scale
Flexibility	BYOK enables organizations to use models they have negotiated pricing or enterprise agreements with
Compliance	BYOK allows organizations to use models within their own data processing agreements (DPAs) with providers
Multi-Provider	BYOK naturally supports multi-provider (OpenAI + Anthropic + Google) without complex credential pooling

ALTERNATIVES CONSIDERED

Alternative	Why Rejected
100% Managed Keys	Platform bears all AI cost risk; cost unpredictability at scale; enterprise customers reject it for compliance; significant billing complexity

Alternative	Why Rejected
100% BYOK Only	Eliminates trial/onboarding capability; high friction for first-time users who haven't sourced keys yet; kills early adoption

IMPACT ANALYSIS

Dimension	Impact
Scalability	BYOK scales linearly with organizations; platform-managed pool is small and capped per-org
Security	BYOK credentials encrypted at rest (AES-256), never returned in plaintext; credential vault is isolated per-org; platform-managed keys managed in a separate high-security secret store
Multi-Tenancy	Each org's BYOK credentials are isolated; platform-managed keys are allocated per-org with strict usage limits
Cost	Platform carries no AI cost for BYOK orgs; platform-managed pool has capped monthly spend per trial org
MVP Impact	BYOK is required at MVP; platform-managed trial keys can be a simple configuration at MVP with a usage cap

DECISION 2 — MULTI-TENANT DATA ISOLATION MODEL

Decision Statement

Determine the database isolation strategy: (A) Shared database with row-level security, (B) Schema-per-tenant, © Database-per-tenant.

Recommended Decision: SHARED DATABASE WITH MANDATORY ORGANIZATION-SCOPED ROW-LEVEL SECURITY + NAMESPACE ISOLATION FOR VECTOR AND OBJECT STORES

FINAL DECISION

The platform uses a **shared relational database** approach with the following mandatory isolation controls:

- 1. Relational Database (Primary):** Single shared database with `organization_id` as a mandatory column on every data table. Row-Level Security (RLS) policies are enforced at the database level (not just application level) as a defense-in-depth measure. Every query must include `organization_id` in the WHERE clause, enforced by the data access layer.
- 2. Vector Store:** Logical namespace isolation per organization (e.g., Qdrant collections are prefixed with `org_{id}_`). No cross-namespace queries are permitted by the API layer or storage layer.
- 3. Object Storage:** Organization-scoped folder/prefix structure with bucket-level policies (e.g., `s3://bucket/org_{id}/documents/`). No cross-organization prefixes are accessible.
- 4. Audit & Execution Logs:** Stored with mandatory `organization_id` partitioning to ensure isolation and enable efficient per-org queries and retention policies.

5. Application-Level Enforcement: Every API endpoint extracts `organization_id` from the authenticated JWT/session and applies it to all queries before reaching the database. This is a non-negotiable architectural requirement enforced via middleware.

WHY THIS IS THE BEST CHOICE

Factor	Reasoning
MVP Feasibility	Schema-per-tenant and DB-per-tenant require provisioning infrastructure per org; shared RLS is immediately manageable for a small engineering team
Operational Simplicity	Single database to manage, backup, migrate, and monitor; no tenant provisioning automation required at MVP
Scalability	Modern databases (PostgreSQL) handle row-level security efficiently; at large scale, can migrate to sharding or schema-per-tenant without changing application logic
Strong Isolation	Database-level RLS + application-level filtering + object storage prefixing provides defense-in-depth
Cost	Shared infrastructure at MVP; upgrade path to dedicated instances for Enterprise tier organizations
Auditability	Single database makes cross-cutting audit queries simpler

ALTERNATIVES CONSIDERED

Alternative	Why Rejected
Schema-per-Tenant	Requires automated schema provisioning per org; migration complexity multiplies with each schema; impractical for MVP team size; adds significant operational overhead
Database-per-Tenant	Maximum isolation but extreme operational overhead; requires automated DB provisioning and management; very expensive per org; suitable only for large enterprises requiring dedicated infrastructure (Phase 3 option)

UPGRADE PATH

At Phase 3, the platform may offer “Dedicated Infrastructure” as an Enterprise add-on, giving high-security organizations a schema-per-tenant or database-per-tenant deployment. The application code remains unchanged because isolation is already enforced via `organization_id` scoping.

IMPACT ANALYSIS

Dimension	Impact
Scalability	Shared DB scales to hundreds of organizations; PostgreSQL RLS is proven at this scale; connection pooling required
Security	Defense-in-depth: DB-level RLS + app-level scoping + storage isolation; any single layer failure does not alone result in data leakage
Multi-Tenancy	Every entity has <code>organization_id</code> ; RLS policies enforce this at the database engine level
Cost	Single database instance; significant cost advantage at MVP scale

Dimension	Impact
MVP Impact	This is the only model feasible for MVP given team size and timeline

MANDATORY ENGINEERING RULES FROM THIS DECISION

1. Every database table MUST have `organization_id` as a non-nullable foreign key
2. Database-level RLS policies MUST be defined for all tenant data tables
3. The data access layer MUST inject `organization_id` into every query from the authenticated context
4. No raw SQL or ORM query may execute against a tenant table without `organization_id` in the WHERE clause
5. Integration tests MUST include cross-tenant isolation verification as part of the test suite

DECISION 3 — VECTOR DATABASE SELECTION

Decision Statement

Select the vector database technology for the knowledge management and RAG system.

Recommended Decision: QDRANT as Primary Vector Store

FINAL DECISION

Qdrant is selected as the platform's vector database with the following rationale and configuration:

- **Self-hosted Qdrant** for production deployments (Docker/Kubernetes)
- **Qdrant Cloud** as a managed alternative for deployments where self-hosting is not preferred
- **Collection naming convention:** `org_{organization_id}_{collection_id}` to enforce tenant isolation at the collection level
- **Embedding model:** Text embeddings generated externally (using the organization's configured AI model provider) before indexing; Qdrant stores vectors only

WHY QDRANT IS THE BEST CHOICE

Factor	Reasoning
Open Source	No vendor lock-in; full control over data; can self-host; no per-query pricing surprises
Multi-Tenant Native	Qdrant supports multiple named collections with independent configuration; natural org-level isolation
Production-Grade	Written in Rust; high performance; stable at production scale
Rich Filtering	Supports payload filtering (metadata) alongside vector similarity search; enables hybrid retrieval in Phase 2
Managed Cloud Option	Qdrant Cloud available for organizations that prefer managed; same API; no code changes
Active Development	Strong community and rapid feature development

Factor	Reasoning
License	Apache 2.0; no commercial licensing concerns
Integration	Well-supported by LangChain, LlamaIndex, and direct REST/gRPC APIs

ALTERNATIVES CONSIDERED

Alternative	Why Rejected
Pinecone	Fully managed / proprietary; per-vector and per-query pricing at scale is unpredictable; no self-host option; vendor lock-in risk
Weaviate	Good option but more complex to deploy; GraphQL query model adds learning curve; heavier resource footprint
Chroma	Excellent for development; not production-hardened for high-concurrency multi-tenant at scale
pgvector (PostgreSQL extension)	Simplifies infrastructure (single database) but has performance limitations at large vector counts; lacks advanced filtering; acceptable as a Phase 1 fallback if operational complexity of Qdrant is prohibitive
Milvus	Powerful but operationally complex; requires Kubernetes and distributed setup; overkill for MVP

FALLBACK OPTION

If Qdrant deployment is not feasible at MVP timeline, **pgvector** (PostgreSQL extension) is the approved fallback. The application's vector storage abstraction layer must hide the specific implementation so switching from pgvector to Qdrant requires no application code changes.

IMPACT ANALYSIS

Dimension	Impact
Scalability	Qdrant scales horizontally; supports sharding; handles millions of vectors per collection
Security	Self-hosted = organization data never leaves controlled infrastructure; collection-level isolation enforces tenant boundaries
Multi-Tenancy	One Qdrant collection per organization knowledge collection; strict naming convention
Cost	No per-query fees; infrastructure cost only (compute + storage); highly cost-efficient
MVP Impact	Qdrant can be deployed as a Docker container for MVP; minimal operational overhead

MANDATORY ENGINEERING RULES

- Vector storage must be accessed through an **abstracted repository interface** — no direct Qdrant SDK calls in business logic
- Collection naming MUST follow: `org_{org_id}_{collection_id}`
- Cross-collection queries are architecturally prohibited

4. Embedding generation is ALWAYS performed by the AI model layer before calling Qdrant
5. All metadata (document_id, chunk_id, page, section, org_id) MUST be stored as Qdrant payload for filtering

DECISION 4 — MVP AI EVALUATION STRATEGY

Decision Statement

Determine the evaluation methodology at MVP: (A) Rule-based only, (B) LLM-as-judge only, © Hybrid.

Recommended Decision: HYBRID EVALUATION — Rule-Based Metrics + LLM-as-Judge via Organization’s Own Model

FINAL DECISION

The MVP evaluation system uses a **Hybrid Evaluation Strategy**:

Layer 1 — Deterministic / Rule-Based Metrics (always active):

- Task completion (did the agent produce a non-empty, non-error response?)
- Response format compliance (does the output match the expected JSON schema or format?)
- Tool call success rate (what percentage of tool calls succeeded vs. failed?)
- Latency measurements (did the agent complete within the configured time budget?)
- Context utilization (did the agent use retrieved knowledge in its response?)

Layer 2 — LLM-as-Judge (optional, activated by organization):

- Uses the **organization’s own configured AI model** (from their BYOK credentials) as the judge
- Evaluates: Groundedness (is the answer supported by retrieved chunks?), Relevance (is the answer relevant to the question?), Completeness (does the answer fully address the question?)
- Evaluation prompts are standardized and version-controlled by the platform
- Cost of evaluation model calls is charged to the organization’s model budget
- LLM-as-judge is configurable per evaluation suite (on/off)

Why this Hybrid approach:

- Rule-based metrics are free, fast, and reliable with no additional model cost
- LLM-as-judge provides nuanced quality assessment that rules cannot capture
- Using the organization’s own model keeps costs transparent and in their control
- Organizations that cannot afford evaluation model calls can still use rule-based metrics

ALTERNATIVES CONSIDERED

Alternative	Why Rejected
Rule-Based Only	Cannot detect semantic quality issues; misses hallucination, poor relevance, or misleading answers

Alternative	Why Rejected
LLM-as-Judge Only	Adds cost on every evaluation run; not all organizations have model credits available; single point of failure if model is unavailable
Platform Pays for Judge Model	Uncontrolled cost; judge model calls scale with evaluation volume; not financially sustainable

IMPACT ANALYSIS

Dimension	Impact
Scalability	Rule-based scales freely; LLM-as-judge scales with org model capacity
Cost	Zero cost for rule-based; transparent, org-borne cost for LLM-as-judge
MVP Impact	Both layers deliverable at MVP; LLM-as-judge implemented as an optional feature

DECISION 5 — PLAN TIER AND USAGE LIMIT STRATEGY

Decision Statement

Define the specific subscription tiers, usage limits, and enforcement strategy.

Recommended Decision: THREE-TIER SUBSCRIPTION MODEL WITH HARD ENFORCEMENT

FINAL DECISION — PLAN TIERS

Feature / Limit	Starter (Free Trial)	Professional	Enterprise
Price Model	Free (14-day trial)	Per seat/month (subscription)	Custom contract
Workflow Executions / Month	500	10,000	Unlimited (fair use)
Concurrent Workflow Runs	5	50	Configurable
AI Agents	5 max	50 max	Unlimited
Knowledge Collections	3 max	25 max	Unlimited
Document Storage	1 GB	25 GB	Configurable
Members	5 max	50 max	Unlimited
Tool Integrations	3 built-in only	20 (built-in + custom)	Unlimited
Audit Log Retention	30 days	1 year	5 years (configurable)

Feature / Limit	Starter (Free Trial)	Professional	Enterprise
Execution Log Retention	30 days	90 days	1 year (configurable)
AI Evaluation	Basic (rule-based)	Full (hybrid)	Full + regression
Analytics	Basic (7-day)	Advanced (90-day)	Full + export
API Access	No	Yes	Yes
Webhooks	No	Yes	Yes
HITL Approvals	Yes (basic)	Yes (full)	Yes (full + delegation)
SSO / SAML	No	No	Yes
SLA	Best effort	99.9% uptime	99.95% + dedicated support
Platform-Managed AI Keys	Yes (metered)	No (BYOK required)	No (BYOK required)

LIMIT ENFORCEMENT POLICY

- 1. Soft Limits (Warning Zone at 80%):** When an organization reaches 80% of any limit, an in-app alert and email notification is sent to the Organization Owner and Admin.
- 2. Hard Limits (Enforcement at 100%):** When an organization reaches 100% of a limit, the relevant action is blocked (new executions are rejected with a clear error; new uploads are rejected). Existing running workflows complete.
- 3. Grace Period:** None by default. Enterprise customers may negotiate a grace buffer in their contract.
- 4. Limit Reset:** Execution and concurrent run limits reset on the monthly billing cycle start date.
- 5. Usage Tracking:** All tracked limits are updated in near-real-time (within 60 seconds of action completion).

IMPACT ANALYSIS

Dimension	Impact
Scalability	Hard limits protect platform from runaway usage; independent limit tracking per org
Multi-Tenancy	Each org's usage is tracked independently; no shared limits
Cost	Clear cost structure; platform can model revenue from usage-based components
MVP Impact	Hard limit enforcement must be in place at MVP to prevent abuse on the trial tier

DECISION 6 — PHASE 2 CONNECTOR STRATEGY

Decision Statement

Define the specific list of Phase 2 integration connectors and their integration approach.

Recommended Decision: 8 PRIORITY CONNECTORS VIA OAUTH 2.0 + CUSTOM HTTP TOOL

FINAL DECISION — PHASE 2 CONNECTOR PRIORITY LIST

Priority	Connector	Category	Integration Method
1	Slack	Communication	OAuth 2.0; webhook notifications
2	HubSpot	CRM	OAuth 2.0; REST API
3	Salesforce	CRM	OAuth 2.0; REST API
4	Zendesk	Ticketing/Support	OAuth 2.0; REST API
5	Jira	Project Management / Ticketing	OAuth 2.0; REST API
6	Google Workspace (Gmail, Drive, Docs)	Productivity	OAuth 2.0; REST API
7	Microsoft 365 (Outlook, Teams, SharePoint)	Productivity	OAuth 2.0; Microsoft Graph API
8	Notion	Knowledge / Docs	OAuth 2.0; REST API

CONNECTOR ARCHITECTURE PRINCIPLES

- OAuth 2.0 Authorization Code Flow:** All connectors use OAuth 2.0 with PKCE where supported
- Token Refresh:** Platform automatically handles OAuth token refresh; connectors must not fail due to expired tokens
- Credential Isolation:** OAuth tokens are stored in the encrypted credential vault, scoped per organization
- Connector Abstraction:** All connectors are implemented behind a unified connector interface; adding a new connector does not change the workflow engine or agent engine
- Tool Node Representation:** Each connector exposes one or more Tool Node types in the workflow builder
- Custom HTTP Tool (MVP):** Organizations can connect to any REST API using the generic HTTP tool; this covers long-tail integrations not in the connector list

WHY THESE 8 CONNECTORS

Connector	Justification
Slack	Highest-demand notification/communication channel; present in nearly all target organizations
HubSpot	Most common CRM for mid-market SaaS companies (primary target segment)
Salesforce	Required for enterprise and larger mid-market customers
Zendesk	Standard support ticketing; customer support automation is top use case
Jira	IT/ops ticketing and project management; IT automation use case
Google Workspace	~60% of target organizations use Google Workspace

Connector	Justification
Microsoft 365	~40% of target organizations use Microsoft 365; covers the other half
Notion	Popular knowledge/doc tool in startups and mid-market tech companies

IMPACT ANALYSIS

Dimension	Impact
Scalability	Connector abstraction allows adding new connectors without system changes
Security	OAuth tokens encrypted at rest; token refresh handled by platform; no user passwords stored
Multi-Tenancy	Each org's OAuth tokens are independently stored and managed
MVP Impact	Phase 2 only; HTTP Tool covers all integrations at MVP

DECISION 7 — APPROVAL “REQUEST CHANGES” BEHAVIOR

Decision Statement

When an approver selects “Request Changes” on an approval request, what happens to the workflow execution?

Recommended Decision: REVISION NODE ROUTING with FALLBACK-TO-FAIL

FINAL DECISION

The “Request Changes” outcome is handled as follows:

Primary Behavior — Revision Node Route:

- If the workflow designer has configured a dedicated “Revision” path branching from the Human Approval Node, workflow execution is routed to that path
- The revision path receives: the approver’s comments, the original proposed action, and the workflow’s current state
- The revision path typically routes to an AI Agent Node that regenerates or revises the output based on the comments, then routes back to a new Human Approval Node
- This creates a **review loop** that continues until the approval is granted or the loop limit is reached

Fallback Behavior — Fail with Reason:

- If no Revision path is configured on the Human Approval Node, “Request Changes” is treated as a **Rejection with Reason**
- The workflow run fails the current step with status “Rejected — Changes Requested”
- The failure reason (approver’s comments) is recorded in the run history
- The organization can re-trigger the workflow with updated inputs based on the feedback

Loop Protection:

- Revision loops have a configurable maximum iteration limit (default: 3 loops)
- If the limit is reached without final approval, the execution fails with “Max revision iterations reached”

WHY THIS DECISION

Factor	Reasoning
Flexibility	Organizations that need structured revision cycles can build them; those that don't don't have to
Simplicity	The fallback-to-fail prevents unexpected indefinite loops for workflows without explicit revision paths
Explainability	Both paths produce clear, auditable outcomes
Safety	Loop limits prevent runaway execution costs from infinite revision cycles

DECISION 8 — AGENT MEMORY STRATEGY

Decision Statement

Define exactly what “stateless,” “session-scoped,” and “long-term” memory mean architecturally, and define the privacy model for long-term memory.

Recommended Decision: THREE-TIER MEMORY MODEL WITH EXPLICIT PRIVACY CONTROLS

FINAL DECISION

Memory Level	Definition	Scope	Storage	Privacy
Stateless	Agent has no memory between invocations. Each run starts fresh with only the workflow context	None	No persistent storage	Maximum privacy; no cross-run data retention
Session-Scoped	Agent retains context within a single workflow run (multi-step tasks). Context cleared at run completion	Single workflow run	Run-level execution state (cleared on run completion)	Cleared automatically; no user data persisted beyond run
Long-Term	Agent retains summarized context across multiple runs for the same triggering entity (e.g., the same customer)	Configurable scope (entity-level)	Dedicated long-term memory store, encrypted per org	Requires explicit admin configuration; subject to data retention policy; PII detection applied before storage

Long-Term Memory Privacy Rules:

1. Long-term memory is **disabled by default**; admins must explicitly enable it per agent
2. Before storing a memory entry, PII detection is applied and detected PII is masked or excluded

3. Long-term memory entries have a configurable TTL (default: 90 days)
4. Long-term memory is scoped to `organization_id + agent_id + entity_key` (e.g., customer email hash)
5. Users may request deletion of their long-term memory entries as part of data subject access requests
6. Long-term memory is stored in the primary relational database (with `organization_id` scoping), not the vector store

DECISION 9 — WORKFLOW IDEMPOTENCY STRATEGY

Decision Statement

How does the platform prevent duplicate workflow executions, and should concurrent runs be the default or opt-in?

Recommended Decision: EXECUTION-KEY-BASED IDEMPOTENCY WITH CONFIGURABLE CONCURRENCY

FINAL DECISION

Idempotency Model:

1. Every workflow execution is assigned an **Execution Key** derived from: `workflow_id + trigger_type + input_hash + time_window`
2. For scheduled and webhook triggers, the time window is the trigger's scheduling interval
3. If a run with the same Execution Key is already in Queued, Running, or Paused state, the new trigger is **deduplicated** and returns the existing run's ID instead of creating a new run
4. This prevents duplicate executions from double-clicks, retry storms, or webhook delivery duplicates

Concurrency Model:

- **Default:** Sequential execution mode — a workflow has only one active run at a time (protected by Execution Key)
- **Opt-in Concurrent Mode:** Workflow managers can enable "Allow Concurrent Runs" per workflow, which disables the Execution Key deduplication and allows multiple simultaneous runs
- This opt-in is appropriate for workflows designed to process independent items in parallel (e.g., document processing)

API Idempotency:

- All state-changing API endpoints support an optional `Idempotency-Key` header
- If the same key is used within 24 hours, the response of the first successful call is returned without re-executing

DECISION 10 — TOKEN COST CALCULATION METHOD

Decision Statement

Should AI token cost be calculated using estimated pricing (using known model pricing tables) or actual billing data from provider APIs?

Recommended Decision: ESTIMATED COST USING PLATFORM-MAINTAINED PRICING TABLE WITH ORG-OVERRIDE CAPABILITY

FINAL DECISION

Cost Calculation Approach:

1. The platform maintains an internal **Model Pricing Table** with the latest known input/output token prices for all supported models
2. Token usage (prompt tokens + completion tokens) is captured from every AI model API response (providers return this data in response headers/body)
3. Estimated cost = $(\text{prompt_tokens} \times \text{input_price_per_1k}) + (\text{completion_tokens} \times \text{output_price_per_1k})$
4. The pricing table is updated by the platform team when providers change prices (manual process; automated monitoring in Phase 2)
5. Organizations with enterprise agreements (custom pricing) can configure a **Custom Price Override** for their specific rates

Accuracy Disclosure:

- The platform clearly labels all cost figures as “estimated” in the UI
- Organizations are advised to verify against their provider billing for financial accounting purposes
- The purpose of in-platform cost tracking is operational awareness, not financial accounting

Why Not Provider Billing APIs:

- Provider billing APIs have significant delays (hours to days)
- Requires OAuth integration with each provider’s billing system; significant engineering overhead
- Different providers have different billing APIs; maintenance burden is high
- Real-time usage feedback (per run) is not possible with delayed billing data

DECISION 11 — REAL-TIME EXECUTION STREAMING

Decision Statement

Should the MVP include real-time streaming of AI agent reasoning steps to the UI, or is final output display sufficient?

Recommended Decision: FINAL OUTPUT AT MVP; STREAMING DEFERRED TO PHASE 2

FINAL DECISION

MVP: The workflow execution monitoring page displays **completed step results only**. Steps are updated in near-real-time via polling (every 2 seconds) or server-sent events (SSE) for run status changes. The full agent reasoning trace is available **after each step completes**.

Phase 2: Real-time token streaming from the AI model to the UI is implemented. Users watching an active run see the AI agent’s reasoning and output stream token-by-token.

Rationale:

- Streaming requires WebSocket or SSE infrastructure with session stickiness; adds significant complexity for MVP
- The core value (seeing what the AI did and why) is delivered by near-real-time polling + post-step trace display
- Most business users (Priya, David from personas) care about the outcome and approval requests, not live token streaming
- Technical users (Arjun from personas) need debugging capability; full trace display after step completion meets this need at MVP
- Streaming is a “nice-to-have” UX enhancement, not a core functional requirement at MVP

DECISION 12 — APPROVAL EMAIL ACTION MODEL

Decision Statement

Should approvers be able to approve or reject directly from the email notification, or must they click through to the platform?

Recommended Decision: SECURE TOKENIZED REDIRECT — CLICK-THROUGH TO PLATFORM REQUIRED

FINAL DECISION

MVP Behavior:

- Approval notification emails contain a single “Review Approval Request” button/link
- Clicking the link redirects the approver to the platform’s approval detail page (authenticated)
- If not logged in, they authenticate first, then are redirected to the specific approval request
- All approval actions (Approve, Reject, Request Changes) are performed on the platform, not via email

Why Not Email-Based Approve/Reject Links:

Concern	Explanation
Security	Email-based approve links can be acted on by anyone who intercepts or forwards the email; bypasses authentication
Context Loss	Approvers need full context (agent reasoning, proposed action, workflow state) to make a quality decision; email cannot display this safely
CSRF Risk	Email action links embedded as GET requests are trivially exploitable
Audit Quality	Platform-based actions have full session context; email-based actions lack user-agent, session, and IP context
Audit Requirement	The BRD requires approval decisions to include actor identity, which requires authenticated platform access

Phase 3 Consideration: Mobile-push notifications with biometrically authenticated in-app approval (native app) may be considered if a mobile app is introduced.

DECISION 13 — DATA RETENTION DEFAULTS

Decision Statement

Define default and maximum data retention periods for each data class.

Recommended Decision: TIERED RETENTION BY DATA SENSITIVITY AND PLAN

FINAL DECISION

Data Class	Starter Default	Professional Default	Enterprise Default	Max Configurable
Workflow Execution Logs	30 days	90 days	1 year	5 years (Enterprise)
Agent Reasoning Traces	30 days	90 days	1 year	5 years (Enterprise)
RAG Retrieval Logs	30 days	90 days	1 year	5 years (Enterprise)
Audit Logs	30 days	1 year	5 years	Permanent (Enterprise contract)
Evaluation Results	30 days	1 year	2 years	5 years
Uploaded Documents	Until deleted	Until deleted	Until deleted	N/A
AI Inputs/Outputs in Trace	30 days	90 days	1 year	5 years
Approval History	1 year	3 years	7 years	Permanent
User Personal Data	Until account deletion + 30 days	Until account deletion + 30 days	Per contract	Per legal obligation
Organization Data	Until org deletion + 30 days	Until org deletion + 30 days	Until org deletion + 90 days	Per contract

Enforcement Rules:

- Automated purge jobs run daily for each organization
- Purge operations are logged as audit events

3. Purge does not break foreign key integrity; soft-deletes used for referenced records
 4. Organizations can shorten retention below their tier's default but cannot exceed it without tier upgrade
 5. Audit logs have a **minimum retention floor** of 30 days regardless of any setting; they cannot be purged before 30 days by any party
-

14. ARCHITECTURE DECISION BASELINE TABLE

This table is the single-page reference for all downstream engineering teams.

#	Decision Area	Final Decision	Key Constraint
1	AI Model Hosting	Hybrid: BYOK Primary + Platform-Managed for Starter	BYOK credentials encrypted; never in plaintext
2	Multi-Tenant Isolation	Shared DB + Mandatory RLS + org-scoped namespaces	<code>organization_id</code> on every entity; DB-level RLS enforced
3	Vector Database	Qdrant (self-hosted / Qdrant Cloud); pgvector fallback	Collection per knowledge collection; prefixed by org_id
4	MVP Evaluation	Hybrid: Rule-based + optional LLM-as-judge (org's model)	Eval cost borne by org; no platform-paid judge model
5	Plan Tiers	Starter (free/trial) / Professional / Enterprise	Hard enforcement at 100%; soft alert at 80%
6	Phase 2 Connectors	8 priority connectors (Slack, HubSpot, Salesforce, Zendesk, Jira, Google Workspace, M365, Notion)	All via OAuth 2.0; HTTP tool covers the rest at MVP
7	Approval Changes	Revision Node route if configured; Fallback = Rejection with reason	Max 3 revision loops (configurable); loop limit prevents runaway cost
8	Agent Memory	3-tier: Stateless / Session / Long-Term; Long-Term opt-in only	PII detection before long-term storage; TTL enforced
9	Idempotency	Execution-Key based deduplication; concurrent mode is opt-in	API Idempotency-Key header support required
10	Token Cost	Estimated via platform pricing table; org custom price override supported	Labeled "estimated" in UI; updated by platform team
11	Streaming	Polling/SSE at MVP; token streaming in Phase 2	2-second polling interval; SSE for status events
12	Approval Email	Click-through to authenticated platform; no email-based actions	Fully authenticated action required for audit compliance
13	Data Retention	Tiered by plan; audit logs minimum 30-day floor	Automated daily purge jobs; purges logged as audit events

15. CANONICAL SYSTEM BOUNDARY DEFINITIONS

These boundary definitions are non-negotiable architectural constraints. Every engineering discipline must design within these boundaries.

15.1 What the Platform IS

- A **workflow orchestration engine** that coordinates AI agents, knowledge retrieval, tool execution, human approvals, and business logic
- A **multi-tenant SaaS application** where each tenant (organization) is fully isolated
- An **AI observability platform** providing traces, metrics, evaluation, and cost tracking for every AI action
- A **governance framework** enforcing risk classification, approval policies, and audit trails
- A **developer platform** (Phase 2) with APIs, webhooks, and extensibility

15.2 What the Platform IS NOT

- NOT an AI model training, hosting, or fine-tuning platform
- NOT a general consumer chatbot or end-user-facing conversational product
- NOT a replacement for core business systems (CRM, ERP, HRMS)
- NOT responsible for the quality or safety of third-party AI models (it governs how they are used)
- NOT a real-time voice/video processing platform

15.3 Mandatory Cross-Cutting Concerns

Every module, service, and API endpoint in the platform MUST:

1. **Enforce `organization_id` scoping** — No data access without valid org context
2. **Validate authentication** — Every request must carry a valid, unexpired auth token (except public registration/login endpoints)
3. **Authorize the action** — Role and permission checks after authentication, before business logic
4. **Log audit events** — Every significant action must produce an immutable audit event
5. **Log errors with context** — Errors must include enough context for debugging without leaking sensitive data
6. **Handle failures gracefully** — All external calls must have timeouts, retries (where safe), and fallback behaviors
7. **Never return credentials in responses** — API keys, model credentials, OAuth tokens are never returned in any response body

15.4 Non-Negotiable Security Rules

1. All inter-service communication must use mutual TLS (mTLS) or equivalent in production
2. No secret, credential, or API key may appear in logs, traces, error messages, or audit events
3. All input from users or external systems must be validated and sanitized before processing
4. Database queries must use parameterized queries/prepared statements; no string-concatenated SQL
5. File uploads must be virus-scanned and format-validated before processing

15.5 AI Safety Boundaries

1. An AI agent MUST NOT execute a tool call of risk level High or Critical without an approved Human Approval record
2. An AI agent MUST NOT access a knowledge collection or tool it is not explicitly permitted to use
3. An AI agent MUST be terminated if it exceeds its configured execution budget (steps, tokens, or time)
4. The orchestration engine is the ONLY component permitted to call AI model APIs for agent execution; no other service bypasses the orchestration layer
5. All AI model API calls must go through the platform's AI Gateway layer which enforces rate limits, cost tracking, and logging

15.6 Workflow Execution Guarantees

1. Every workflow run state must be persisted to durable storage before execution begins
2. Workflow state transitions must be idempotent at the state machine level
3. A workflow step that has been successfully completed must not be re-executed on retry
4. Cancellation of a running workflow must propagate to all in-progress sub-operations within a configurable timeout (default: 30 seconds)
5. The system must be able to recover in-progress workflow runs after an unexpected restart

End of Architecture Decision Baseline

Document Version: 1.0.0 | Status: FINAL This document is the definitive reference for all architectural, database, API, security, and deployment decisions. Any deviation requires a formal Architecture Decision Record (ADR) amendment.